

z/OS Job Control Language (JCL)

SEQUENTIAL DATA SETS AND THE DATA DEFINITION STATEMENT





DATA SETS AND FILES

The main function of a batch system is to process files. Therefore, the DD statement – *Data Definition* – is the most important JCL statement. To use it effectively, we need to understand how z/OS manages files.

IBM mainframes developed independently from other types of systems, such as Unix, and the approach to handling files is different.

In some respects the differences are functional. In others the differences are mainly a question of terminology.

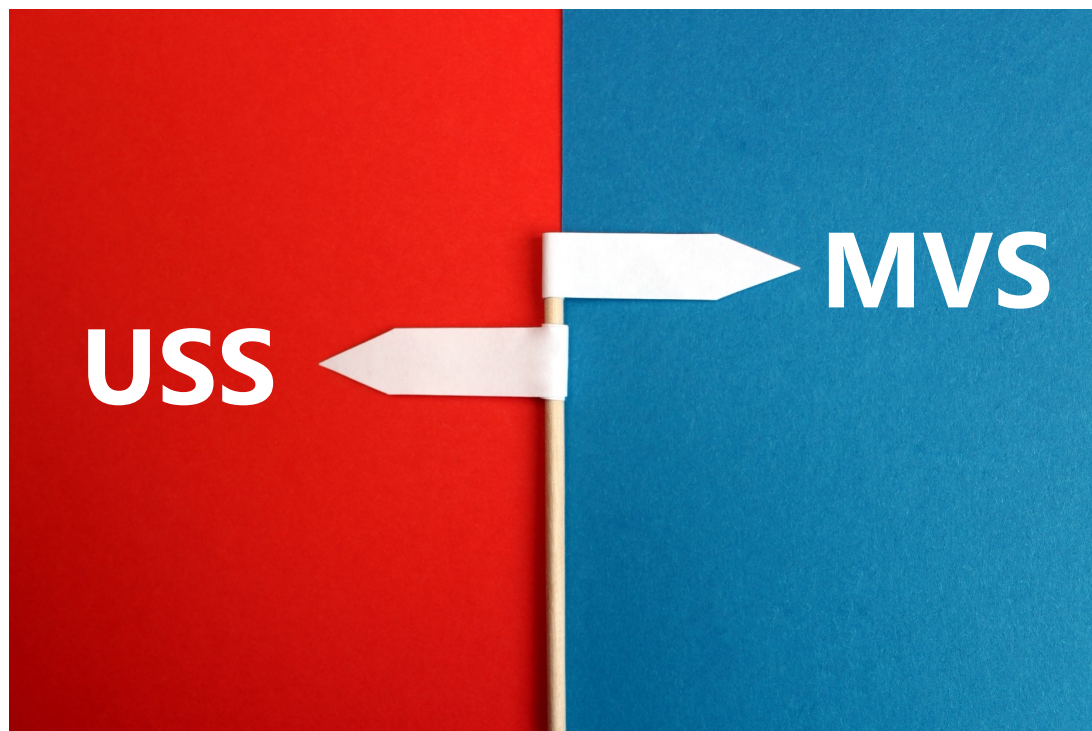
We'll attempt to clarify all that in this module.



DATA SETS AND FILES

TWO SYSTEMS IN ONE...SORT OF

z/OS



It can be challenging to describe how z/OS manages files because the system supports both the traditional mainframe approach and POSIX.

Mainframers often speak of z/OS as having an "MVS side" and a "Unix side," although it is all one operating system.

MVS stands for Multiple Virtual Storage, the name of an ancestor of present-day z/OS.

POSIX functionality on z/OS is branded as Unix System Services (USS), but it is not implemented as a separate Unix instance.

Both the implementation and the terminology for files are different on each "side" of z/OS.

Over time, the terminology has bled over in both directions, and many people use imprecise language to describe z/OS file management.

USS AND MVS DIFFERENCES IN FILE MANAGEMENT



File

A named collection of data items stored in a consistent format.

File are either character streams or byte streams.

Logical records in character files are delimited by *new line* characters embedded in the data.

Data Set

A named collection of data items stored in a consistent format.

Several data set formats are used. The internal organization of data is different in each.

Logical records are formatted as defined for each file type. They may or may not include embedded control codes, but they never contain new line characters.

USS AND MVS DIFFERENCES IN FILE MANAGEMENT



File System

A set of data structures and algorithms to manage file access. Examples: zFS, HFS.

Storage devices (disks, solid state, etc.) must be formatted for a particular file system before files can be stored there.

Access Method

An algorithm to manage file access. Examples: QSAM, BPAM, VSAM.

Storage devices (disks, solid state, etc.) are not pre-formatted to support an access method.

When a data set is created, it is formatted to work with a particular access method.

Data sets of different types may reside on the same storage device (volume).

USS AND MVS DIFFERENCES IN FILE MANAGEMENT



Inode

A data structure containing metadata about a file or directory, including its location.

Inode Table

Data structure in a file system that keeps track of metadata and locations of files and free space that the file system manages.

System Catalog

Data structures and logic to keep track of metadata and locations of data sets.

VTOC

Volume Table of Contents – data structures residing on each volume that keep track of used and available space and fragmentation. DASD volumes must be formatted with a VTOC before use.

USS AND MVS DIFFERENCES IN FILE MANAGEMENT



Nested Directories

A logical hierarchy of "folders" that contain files.

System security is based on the *mode* of both directories and individual files that are logically "under" the directories (or "in" the folders).

The naming convention can be helpful to organize files in a sensible way.

Qualified Names

Dot-separated data set names consisting of a high-level qualifier, one or more mid-level qualifiers, and a low-level qualifier.

System security is based on the hierarchy of qualifiers (to an extent).

The naming convention can be helpful to organize data sets in a sensible way.

USS AND MVS DIFFERENCES IN FILE MANAGEMENT



Home Directories

Each defined user id has a home directory where they can create files to use in their work.

High-Level Qualifier

The system user id is the high-level qualifier for data sets the user creates and uses in their work.

USS AND MVS DIFFERENCES IN FILE MANAGEMENT



Space Allocation

Once defined, a file can grow until the storage unit runs out of space to store it.

Space Allocation

Space for a data set is defined when the data set is created.

The *primary extent* is the amount of space allocated initially.

The *secondary extent* is the quantity of space the system adds when the primary allocation is used up.

A data set can "run out" of space before the volume is full, if it exceeds its primary and secondary extents.

USS AND MVS DIFFERENCES IN FILE MANAGEMENT



User Interaction

Users use a command line facility, either **omvs** or **ishell** (or both) to work with their USS files.

The shells are accessed through TSO/ISPF.

User Interaction

Users use TSO commands and ISPF panels to work with their MVS data sets.

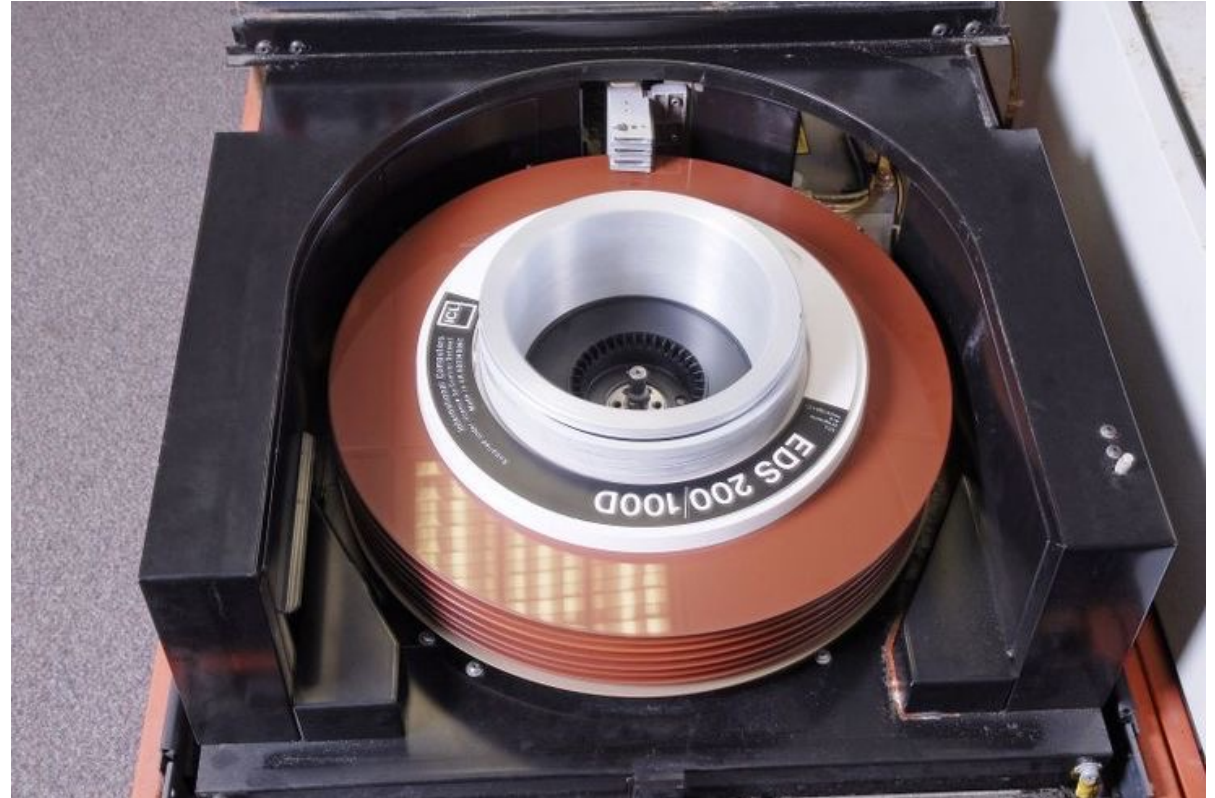
DIRECT ACCESS STORAGE DEVICES (DASD)

DASD – DIRECT ACCESS STORAGE DEVICE

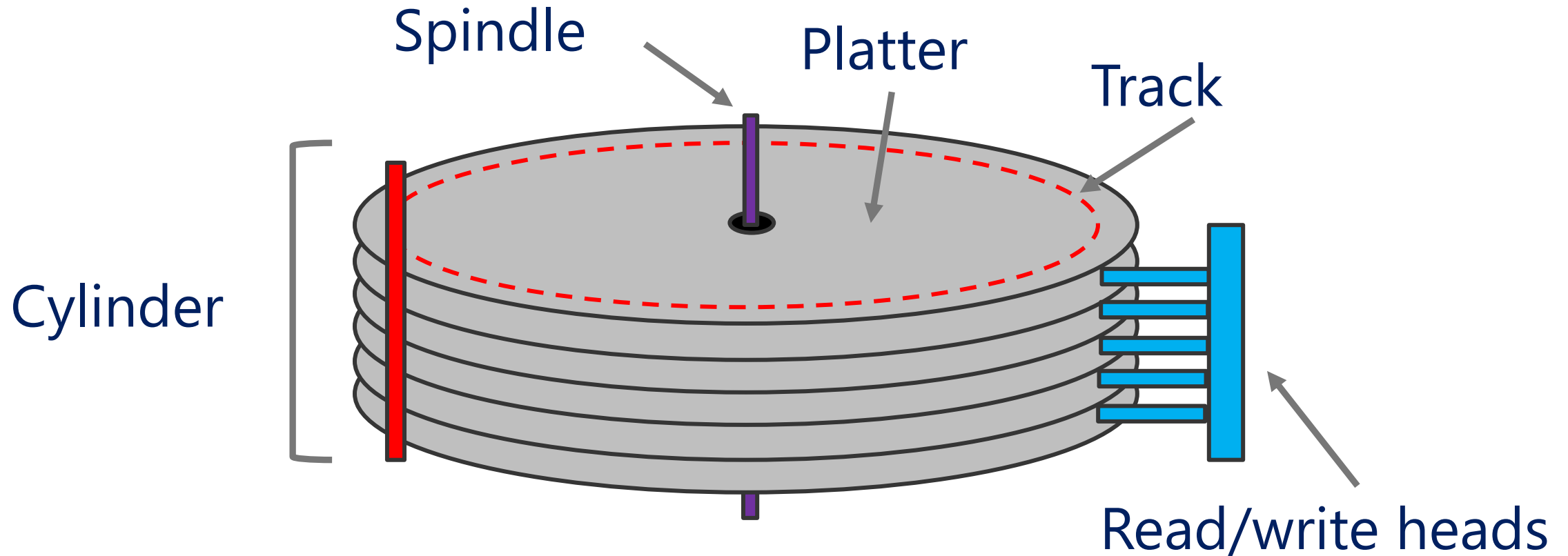
All peripheral devices are virtualized in z/OS. The physical data storage device may be anything – optical, solid state, holographic, or whatever comes next.

z/OS sees data sets as residing on Direct Access Storage Devices, or DASD, as if it were interacting with 1970s technology (the brain in a jar).

Therefore, when we write DD statements to create MVS data sets, we specify the characteristics of the data sets as if they resided on 1970s-era DASD equipment,.



DASD – DIRECT ACCESS STORAGE DEVICE



z/OS sees this technology when accessing data sets, regardless of the technology actually in use.

DATA SET TYPES, ACCESS METHODS, OPERATIONS

DATASET TYPES, ACCESS METHODS, OPERATIONS



The following slide shows a summary of the various data set types, the access methods that apply to them, and the read and write operations that are supported for each.

The summary is provided for reference. You don't have to memorize the information.

The dataset types and access methods you will use in general application development are

- Physical Sequential – QSAM
- Partitioned Extended (PDSE) – BPAM
- VSAM types – ESDS, KSDS, and RRDS

DATA SET TYPES, ACCESS METHODS, OPERATIONS

	DD	DD	DD	Access			
	DSORG	DSNTYPE	RECOrg	Method	Read	Write	Notes
Physical Sequential	PS			BSAM	F	O,A	by block
Physical Sequential	PS			QSAM	F	O,A	by logical record
Direct Access	DA			BDAM	D	D	(backward compat.)
Partitioned (PDS)	PO			BPAM	F	O,A	per member
Partitioned (PDSE)	PO	LIBRARY		BPAM	F	O,A	per member
VSAM ESDS	VS		ES	VSAM	F,D	O,A,D	direct by RBA
VSAM RRDS	VS		RR	VSAM	F,B,S,D	O,A,D	direct by RRN
VSAM KSDS	VS		KS	VSAM	F,B,S,D	O,D	direct by RID
VSAM LDS	VS		LS	VSAM	F	A	byte stream
Indexed Sequential	IS			ISAM	F,B,S,D	D	(obsolete)
Undefined	U			(none)			for binary data

F=Forward, B=Backward, S=Skip Sequential, D=Direct, O=Overwrite, A=Append

THE SYSTEM CATALOG

THE SYSTEM CATALOG

Terminology:

- allocate – create data set
- unallocate –delete data set
- catalog – add data set information to catalog
- uncatalog – remove data set information from catalog.
- volume – logical Direct Access Storage Device (DASD).

Data sets may be catalogued or uncatalogued.

To reference an uncatalogued data set we must tell JES the volume(s) where the data set resides.

To reference a catalogued data set we only need to tell JES the data set name. Other information about the data set is in the catalog.

THE DATA DEFINITION (DD) STATEMENT

OPERATION: DD

```
//MYFILE DD DSN=HLQ.INNOV.SAMP01,  
//      DISP=... ,  
//      DCB=...
```

The DD operation identifies this JCL statement as a Data Definition statement.

DDNAME

```
// MYFILE DD DSN=HLQ.INNOV.SAMP01 ,  
//      DISP=... ,  
//      DCB=...
```

The DDNAME matches the name specified in the application program to refer to this data set.

DSN OR DSNAME

```
//MYFILE DD DSN=HLQ.INNOV.SAMP01,  
//      DISP=... ,  
//      DCB=...
```

The Data Set Name matches the catalogued name of the data set, or the data set name in the VTOC if not catalogued.

Between them, the DDNAME and DSNAME connect the data set to the program executed in the job step.

DISP

```
//MYFILE DD DSN=HLQ.INNOV.SAMP01 ,  
//      DISP=(NEW,CATLG,DELETE) ,  
//      DCB=...
```

The disposition parameter tells JES how to open the dataset and what to do with it when the job step completes.

DISP – STATUS

DISP= (SHR , ,)
OLD
MOD
NEW

The first subparameter is the status of the data set.

The JES initiator will enqueue on the data set, requesting exclusive (EXCL) or shared (SHR) access based on the status.

DISP – STATUS

DISP= (SHR , ,)
OLD
MOD
NEW

Status SHR means other jobs can access the data set while this job step uses it.

DISP – STATUS

DISP= (SHR , ,)
OLD
MOD
NEW

Status OLD requests exclusive access to the data set for the duration of the job step.

DISP – STATUS

DISP= (SHR , ,)
OLD
MOD
NEW

Status MOD means the application program will append data to the end of the sequential data set.

DISP – STATUS

DISP= (SHR , ,)
OLD
MOD
NEW

Status NEW means the job step will create the data set. If the data set already exists, the job step will fail.

NEW is the default value for status, if none is specified.

DISP – DISPOSITION ON SUCCESSFUL STEP COMPLETION

DISP= (, KEEP ,)
DELETE
PASS
CATLG
UNCATLG

The second subparameter of DISP controls the fate of the data set on successful completion of the job step.

For data sets that existed when the step started, the default is KEEP. For new data sets, the default is DELETE.

DISP – DISPOSITION ON STEP FAILURE

**DISP= (, , KEEP)
DELETE
CATLG
UNCATLG**

The third subparameter of DISP controls the fate of the data set when the job step fails – that is, completes with a condition code value other than zero.

DISP

```
//MYFILE DD DSN=HLQ.INNOV.SAMP01 ,  
//      DISP=(NEW,CATLG,DELETE) ,  
//      DCB= . . .
```

This example means the data set will be created in this job step. If the job step completes successfully, the data set will be catalogued. If the job step fails, the data set will be deleted.

DCB PARAMETER

```
//MYFILE DD DSN=HLQ.INNOV.SAMP01,  
//      DISP=... ,  
//      DCB=...
```

The DCB parameter provides values for the Data Control Block, the data structure that represents a data set in an application program or in system code.

It is specified for uncatalogued data sets, to provide the information usually stored in the catalog entry. Typically, these are new data sets to be allocated in the job step.

DCB PARAMETER

```
DCB= (DSORG=PS ,  
      RECFM=FB ,  
      LRECL=100 ,  
      BLKSIZE=2000)
```

There are quite a few options for the DCB parameter, and many are specific to particular data set types. For now, we will use only a handful that are relevant to physical sequential data sets.

- **DSORG** Data Set Organization – Physical Sequential
- **RECFM** Record Format – Fixed Blocked
- **LRECL** Logical Record Length – 100
- **BLKSIZE** Block Size – 2000

SPACE PARAMETER

```
//MYFILE DD DSN=HLQ.INNOV.SAMP01 ,  
//      DISP=... ,  
//      SPACE=...
```

The SPACE parameter tells JES how much DASD space to allocate for a new data set.

SPACE can also be used to override the value in a data set's catalog entry for the duration of the job step, usually to increase the secondary space allocation temporarily.

SPACE PARAMETER

`SPACE=unit (primary, secondary)`

There are many possible options for coding the SPACE parameter.

For now, we will limit ourselves to a few basic values.

The unit can be TRK (tracks), CYL (cylinders), and a few others.

The primary extent is the amount of DASD space to allocate to the data set initially. The secondary extent is the amount of space to add when the primary extent becomes full.

PHYSICAL SEQUENTIAL DATA SETS

LOGICAL RECORDS AND BLOCKS

A "logical record" is what your application considers a record in a file.

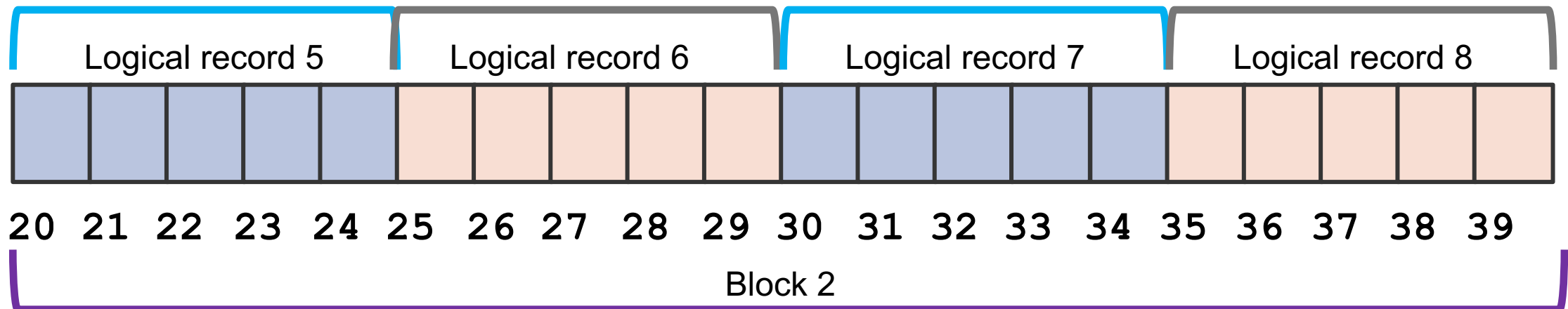
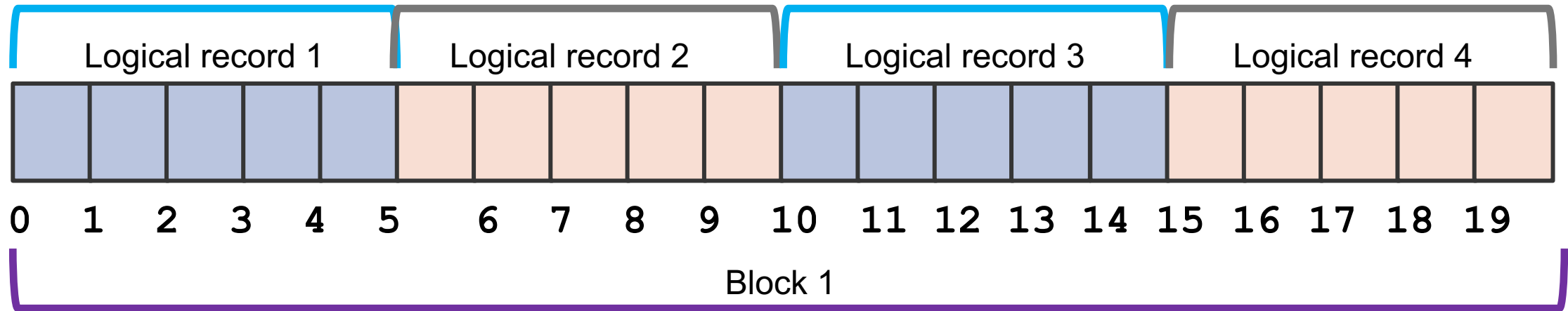
A block contains one or more logical records.

A block is the unit of data for I/O transfers. It is also called a "physical record".



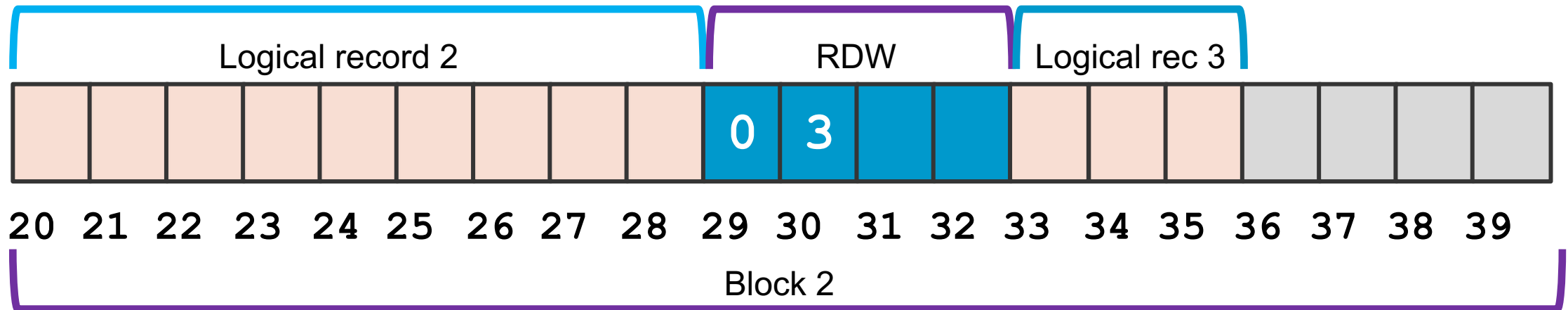
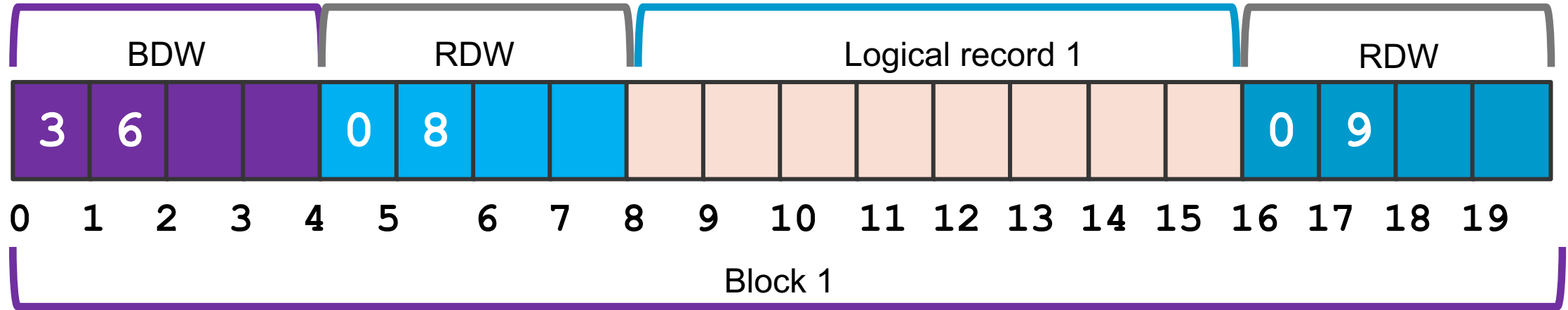
LOGICAL RECORDS AND BLOCKS – FIXED LENGTH

DCB= (DSORG=PS , RECFM=FB , LRECL=5 , BLKSIZE=20)

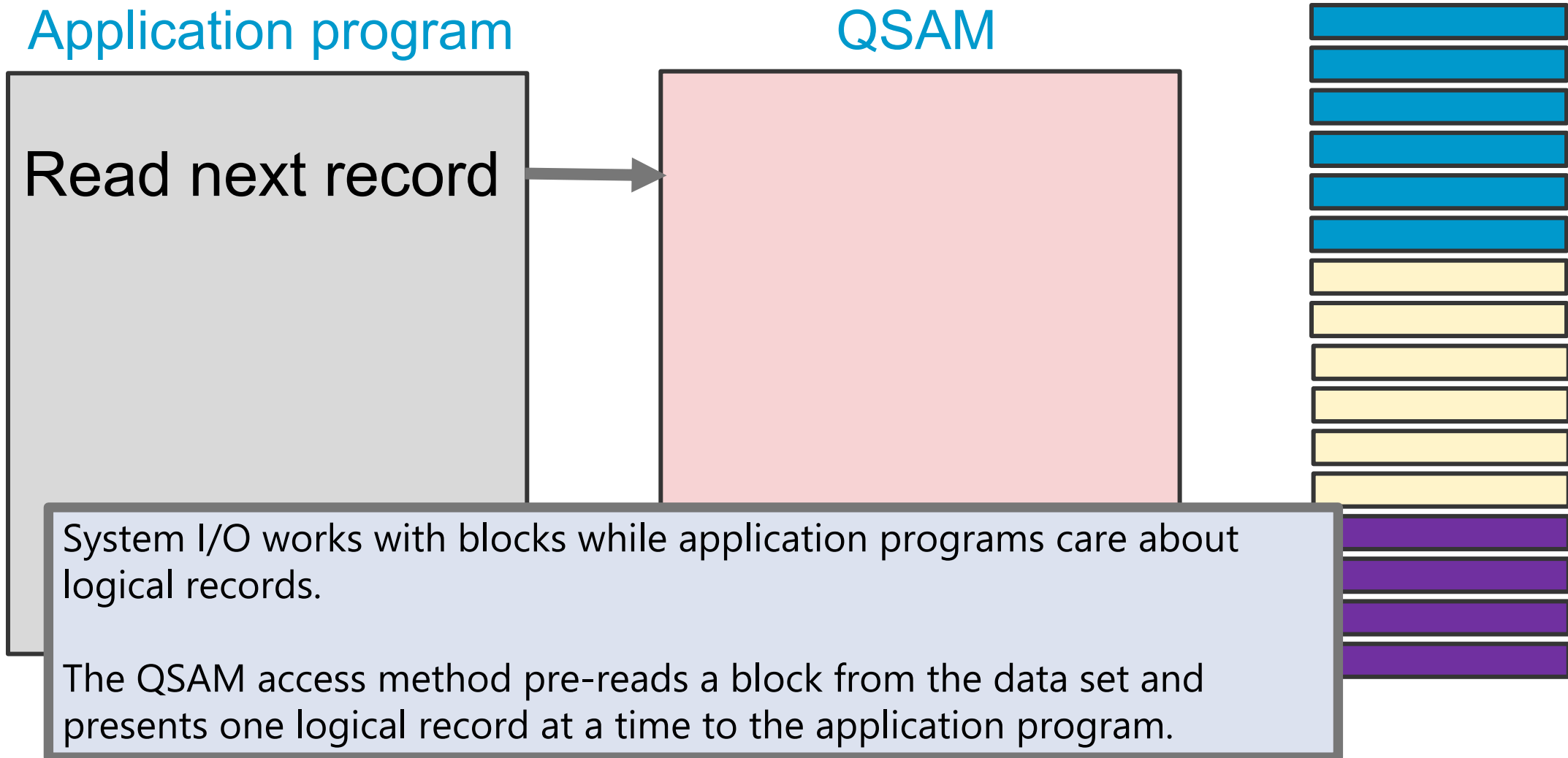


LOGICAL RECORDS AND BLOCKS – VARIABLE LENGTH

DCB= (DSORG=PS , RECFM=VB , LRECL=6 , BLKSIZE=40)



QUEUED SEQUENTIAL ACCESS METHOD (QSAM)



QUEUED SEQUENTIAL ACCESS METHOD (QSAM)

Application program

Read next record

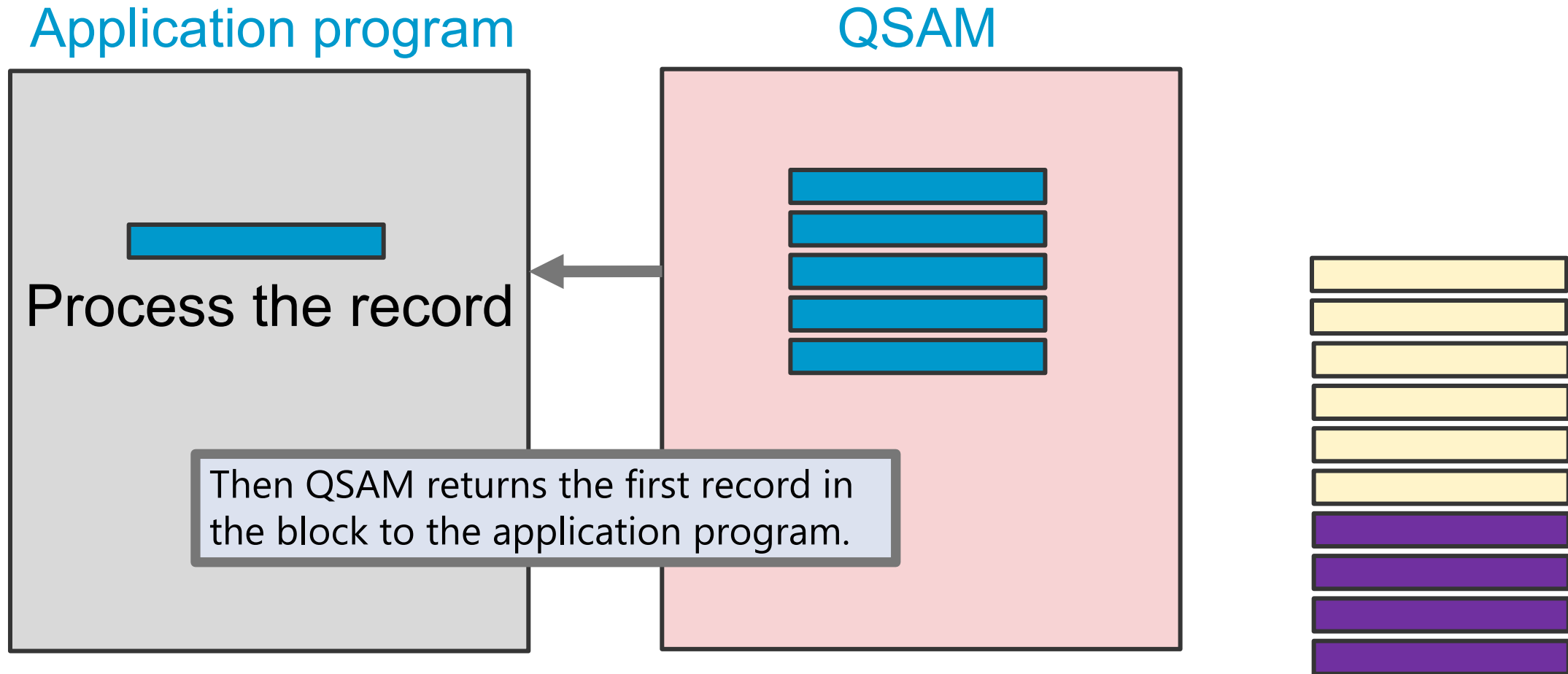
QSAM



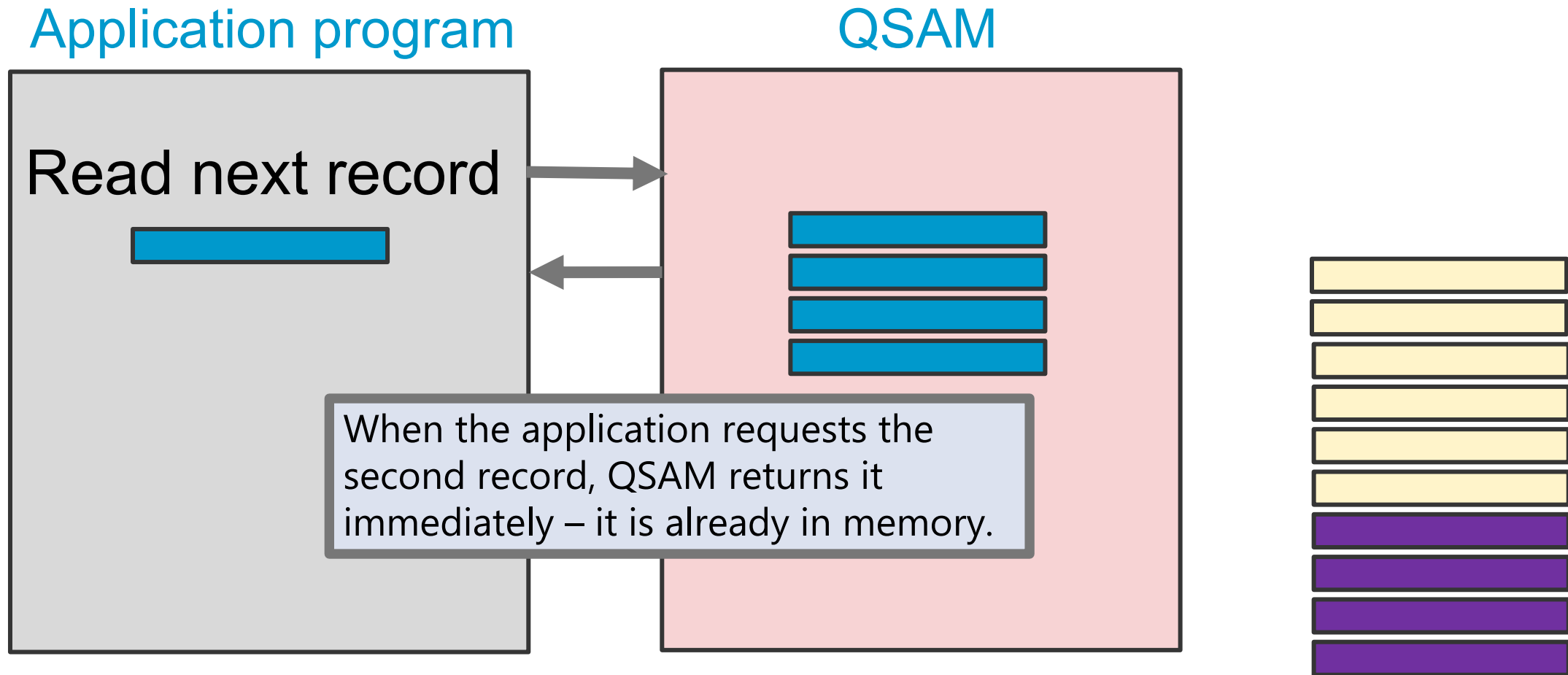
When the application program requests the first record, QSAM reads the first block and queues it in anticipation of more sequential read requests from the application.



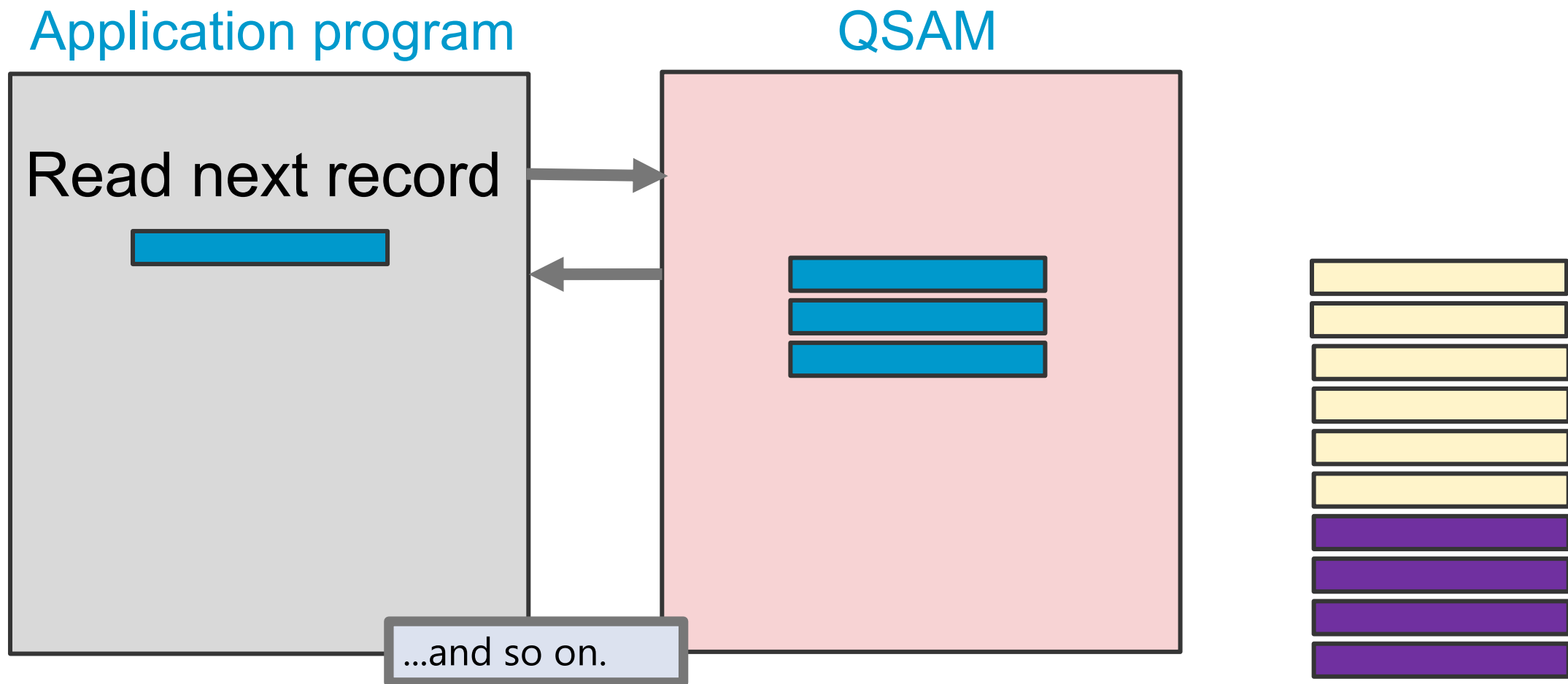
QUEUED SEQUENTIAL ACCESS METHOD (QSAM)



QUEUED SEQUENTIAL ACCESS METHOD (QSAM)



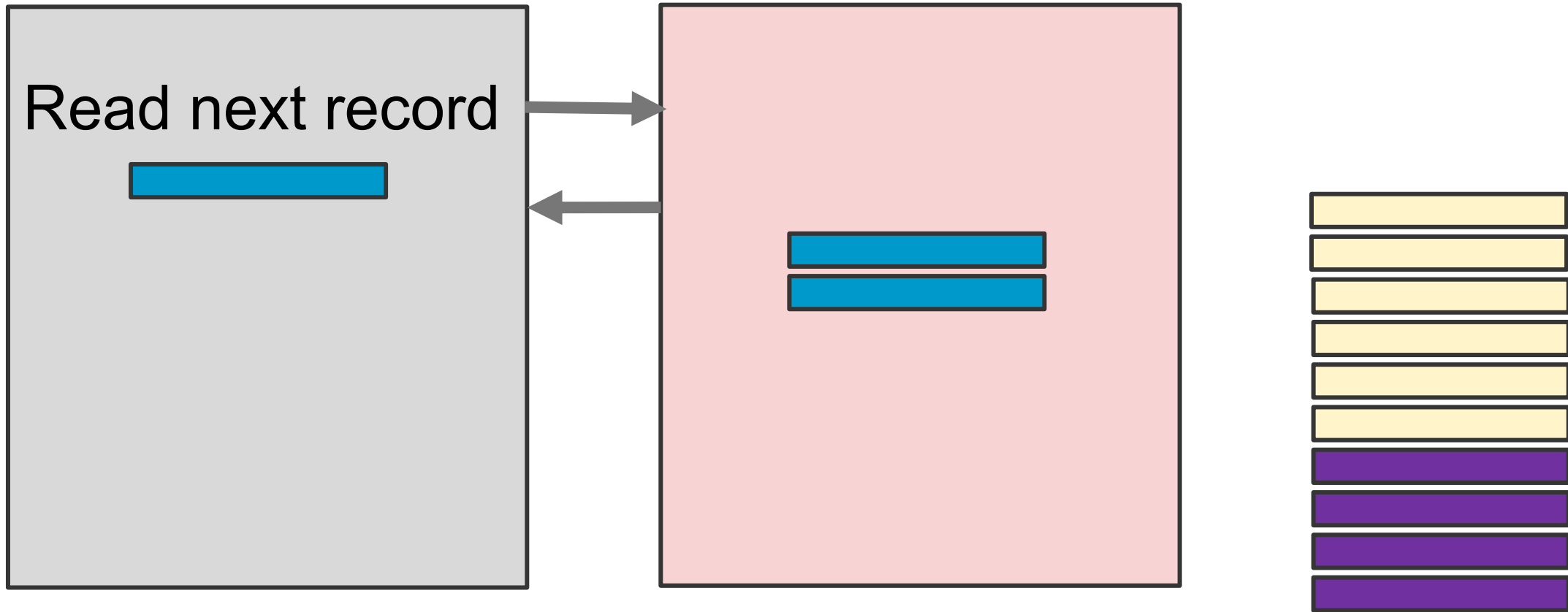
QUEUED SEQUENTIAL ACCESS METHOD (QSAM)



QUEUED SEQUENTIAL ACCESS METHOD (QSAM)

Application program

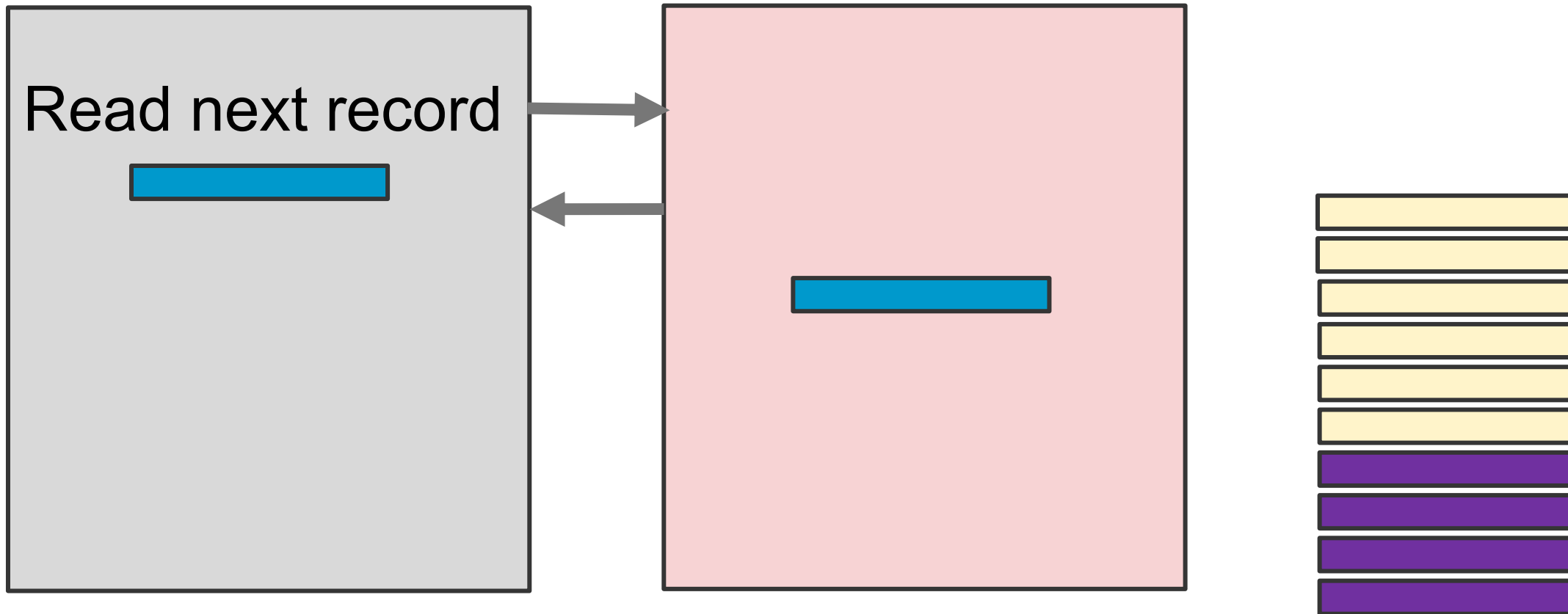
QSAM



QUEUED SEQUENTIAL ACCESS METHOD (QSAM)

Application program

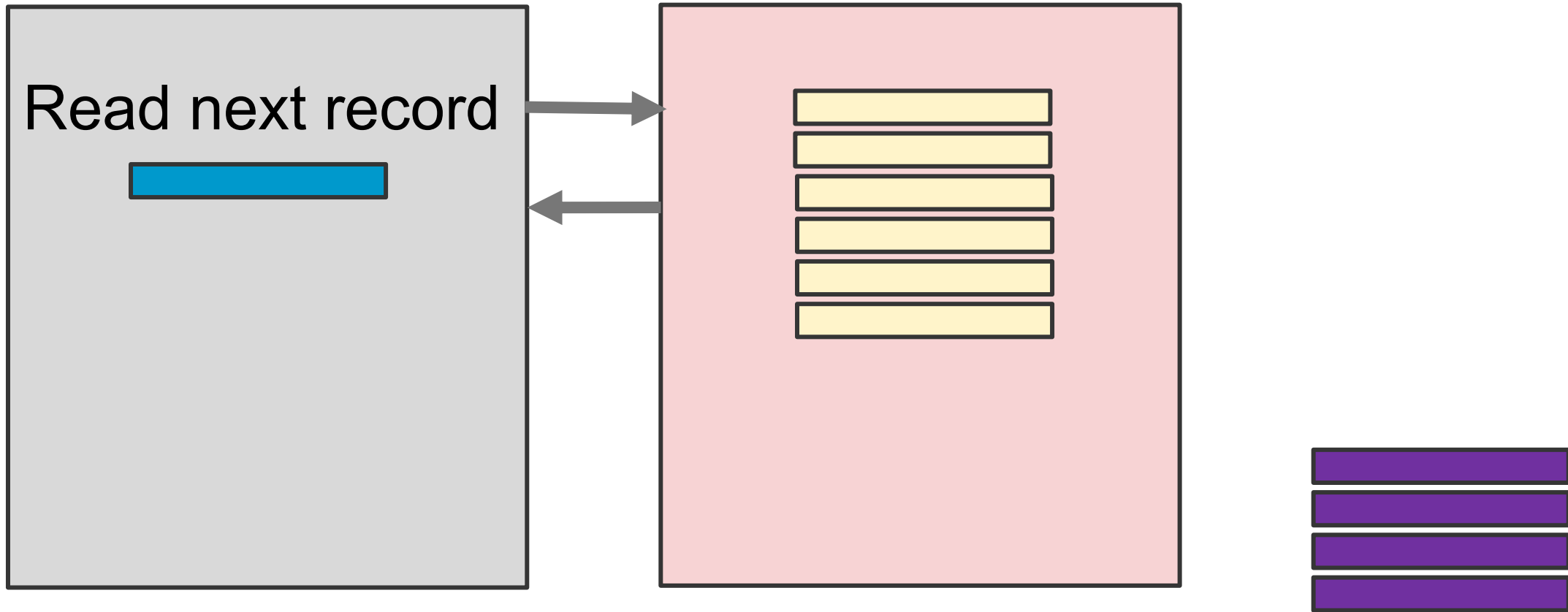
QSAM



QUEUED SEQUENTIAL ACCESS METHOD (QSAM)

Application program

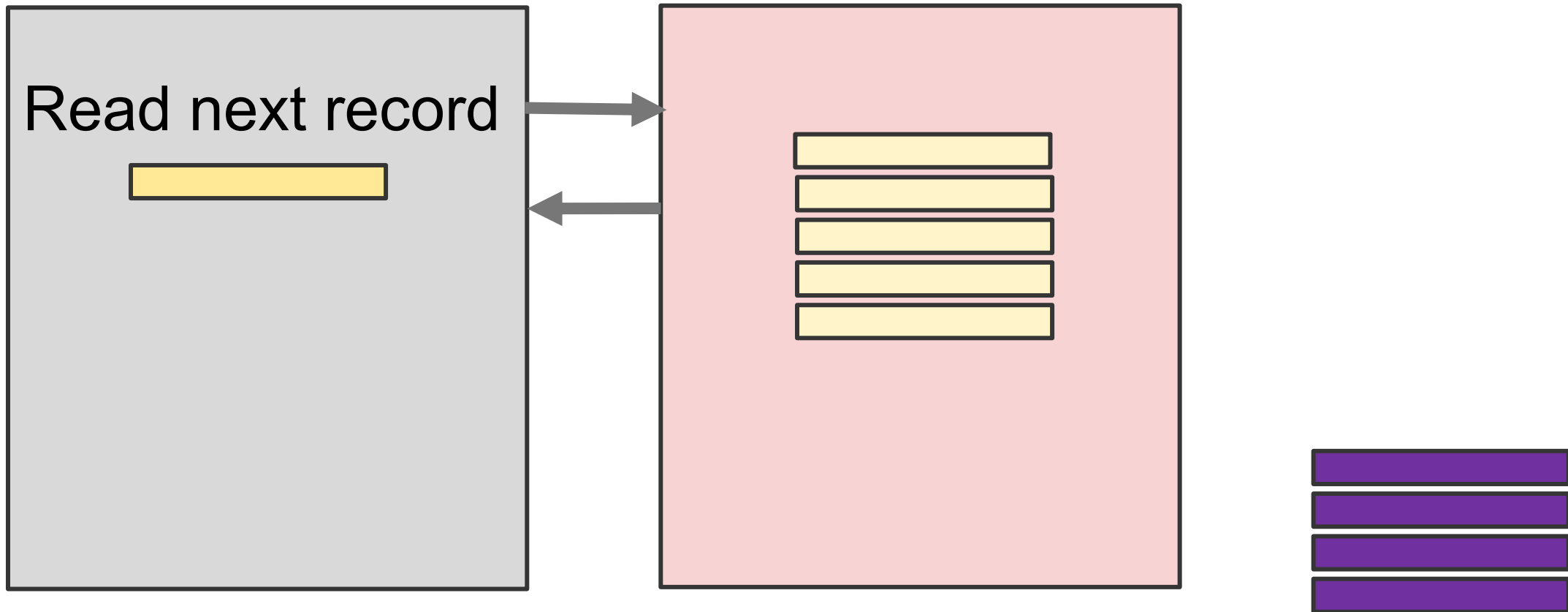
QSAM



QUEUED SEQUENTIAL ACCESS METHOD (QSAM)

Application program

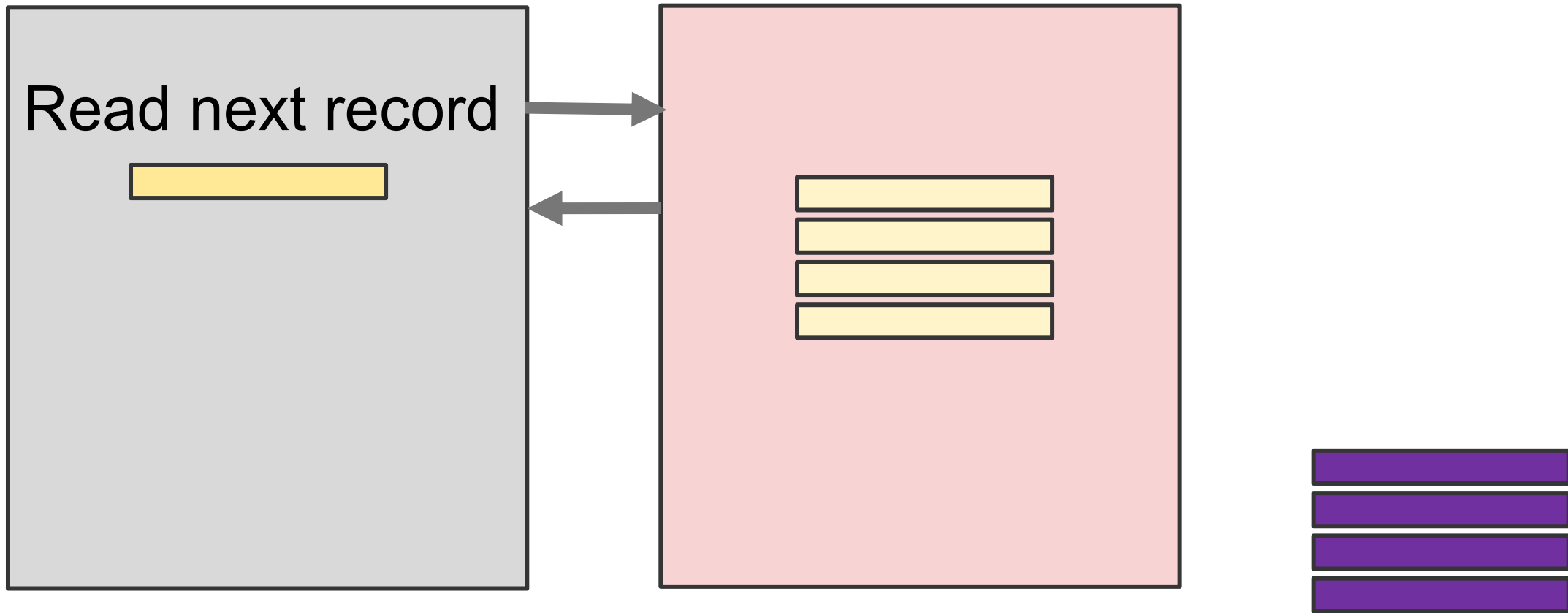
QSAM



QUEUED SEQUENTIAL ACCESS METHOD (QSAM)

Application program

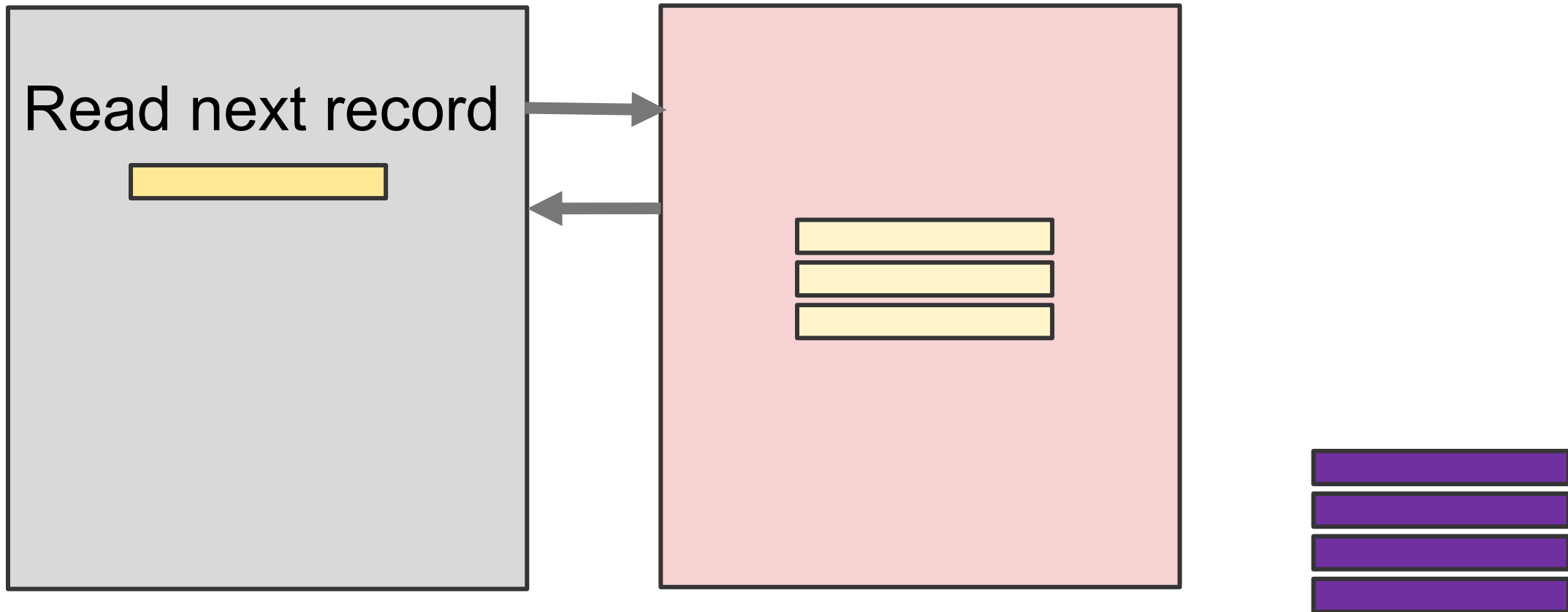
QSAM



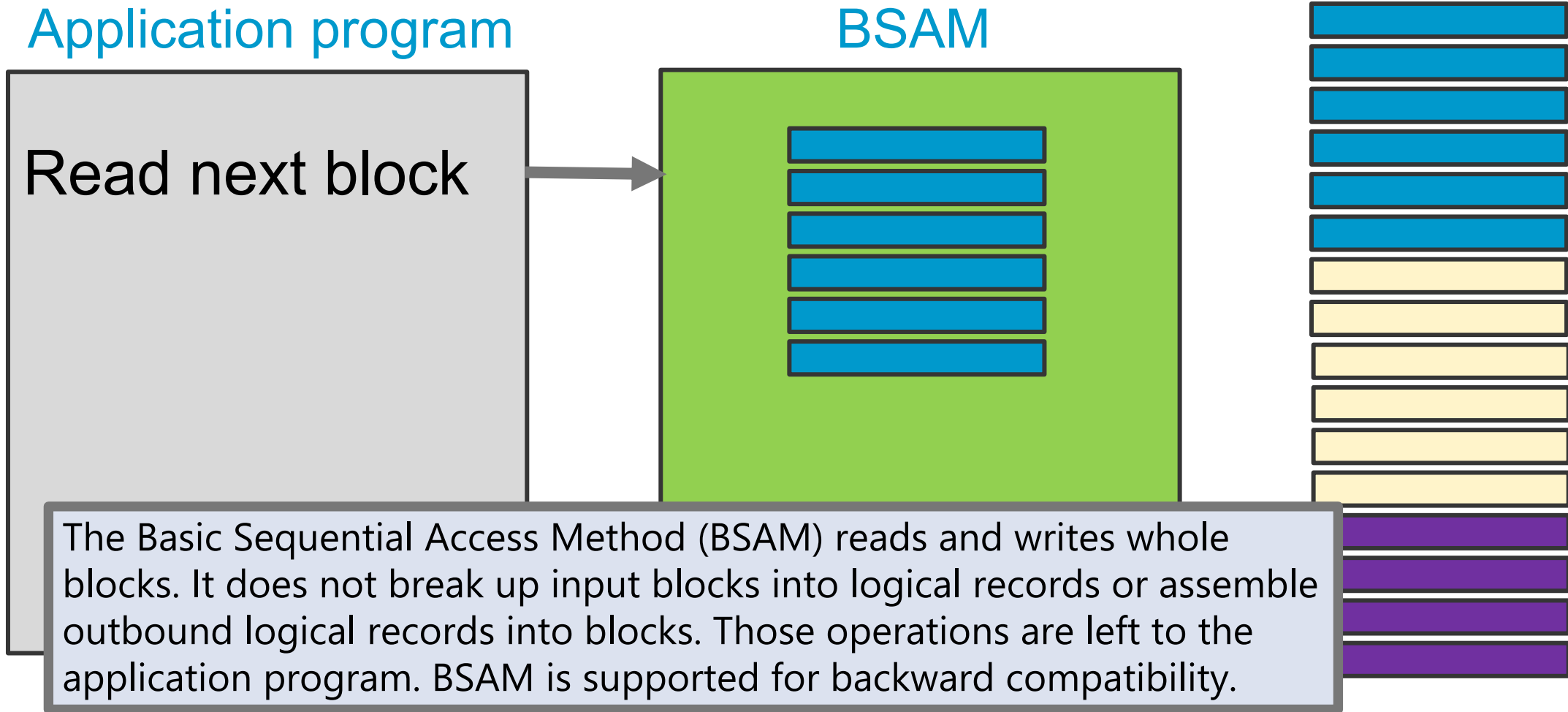
QUEUED SEQUENTIAL ACCESS METHOD (QSAM)

Application program

QSAM



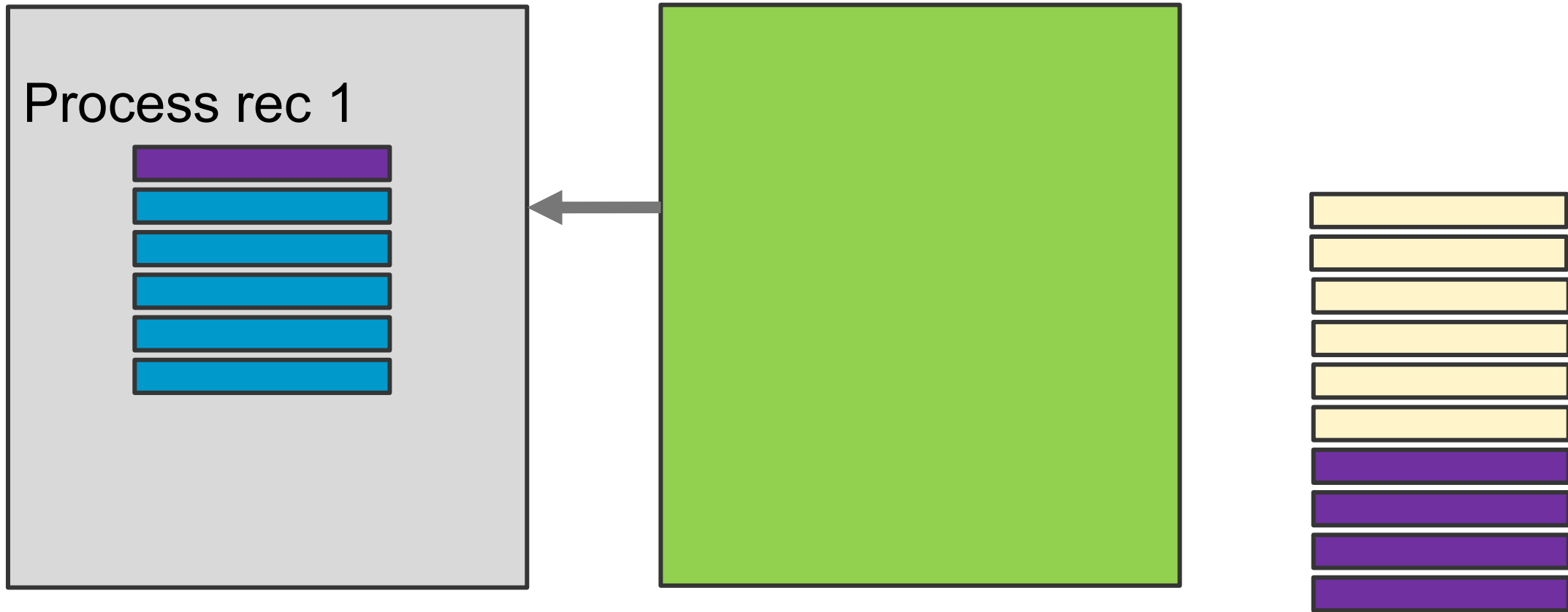
BASIC SEQUENTIAL ACCESS METHOD (BSAM)



BASIC SEQUENTIAL ACCESS METHOD (BSAM)

Application program

BSAM



BASIC SEQUENTIAL ACCESS METHOD (BSAM)

Application program

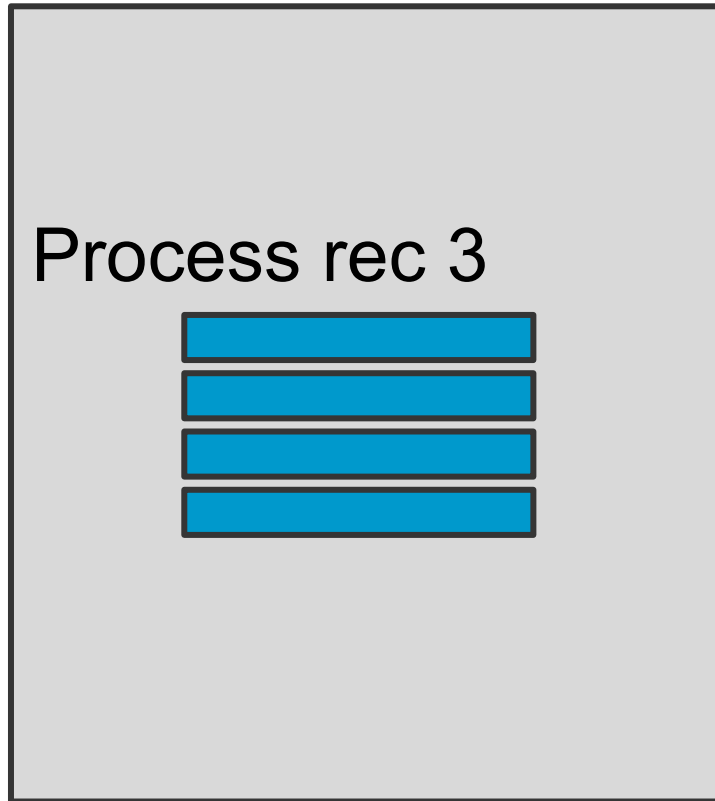


BSAM



BASIC SEQUENTIAL ACCESS METHOD (BSAM)

Application program

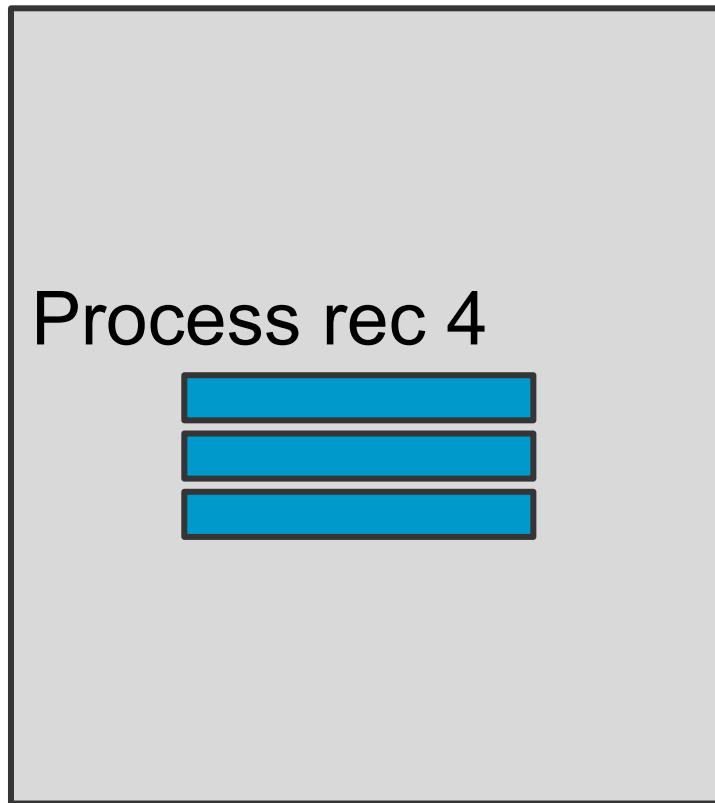


BSAM



BASIC SEQUENTIAL ACCESS METHOD (BSAM)

Application program



BSAM



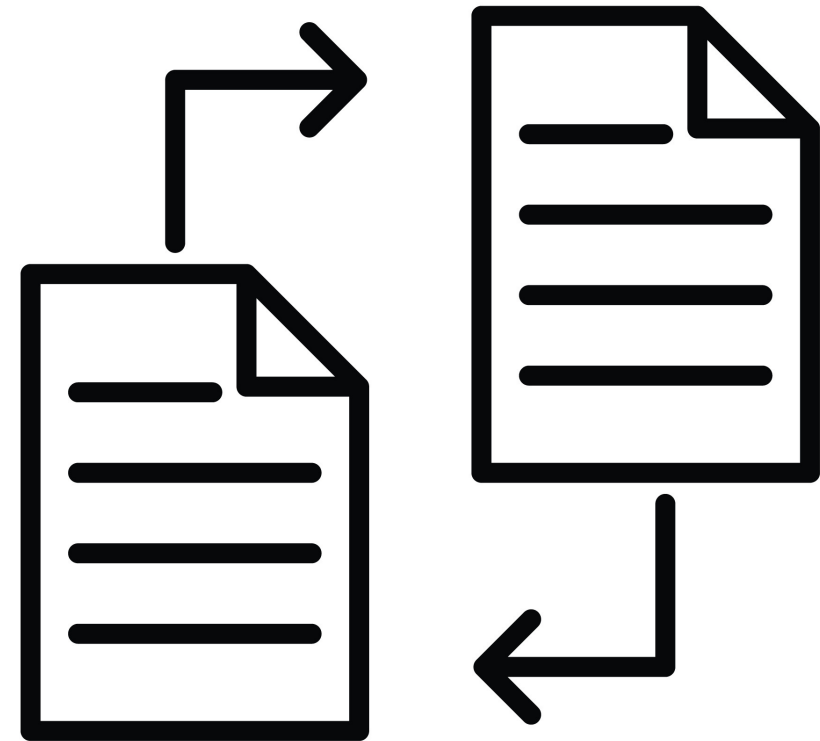
CONVERSIONS DURING FILE TRANSFER

When you transfer files between the Control Node and the z/OS Managed Node, the data must be converted into a form the receiving system can work with.

The obvious translation is between EBCDIC, the native text encoding of z/OS, and ASCII, the native text encoding for most other systems.

Now that you've seen how some of the z/OS data set types are structured internally, you know that the embedded control codes that define block sizes and variable record lengths have to be removed when a data set is transferred to Linux, and newline characters must be inserted at the end of each logical record. The reverse translations have to occur when transferring a file to z/OS.

Your 3270 terminal emulator and the `ibm_zos_collection` handle these translations for us. FTP also handles them.



IEFBR14: THE DO-NOTHING UTILITY

THE VALUE OF DOING NOTHING



You might think the "Hello, World!" sample program does practically nothing at all, and yet there are times when we need a program that does even less.

Why?

JES can do a lot with data sets based on DD statements alone. The problem is that JES can't tell what we're trying to do if we code only DD statements with no EXEC statement.

Consider a job step whose purpose is to create some data sets to be used in the following step. The next step runs an application program that writes to those data sets. How can JES tell which step is which if there's no EXEC statement?

So, we need an EXEC statement...but we don't have any application processing to perform.

Enter the "do-nothing" utility, IEFBR14.

THE VALUE OF DOING NOTHING

IEFBR14 immediately returns to the caller – that is, to z/OS – without performing any further work.

IEF is the three-letter product code for the utility. The rest of the name, BR14, comes from the Assembler instruction used to return to the calling program: BR 14, or "branch on register 14."

According to the z/OS standard linkage convention, general register 14 holds the return address in the program that invoked the present program.

So, IEFBR14 enables JES to act on the specifications in the DD statements in the job step.



WHAT THE DO-NOTHING PROGRAM DOES

```
*****
* MY VERY OWN IEFBR14 PROGRAM. IT DOES NOTHING, BUT DOES IT MY WAY.
*****
MYBR14  CSECT
MYBR14  AMODE 31
MYBR14  RMODE ANY
        STM  R14,R12,12(R13)      TYPICAL PROGRAM ENTRY CODE
        LR   R12,R15
        USING *,R12
*
*                                     NO APPLICATION LOGIC AT ALL
*
        SR   R15,R15              TYPICAL PROGRAM EXIT CODE
        LM   R14,R12,12(R13)
        BR   R14
R12      EQU  12
R13      EQU  13
R14      EQU  14
R15      EQU  15
        END  MYBR14
```

Note: This is "good to know" but not necessary for using Ansible to manage z/OS nodes.

Since the audience for this training consists of people familiar with Unix, Linux, and Windows, it might be worthwhile to explain that z/OS does not have a stack and heap.

Instead, it uses a convention for invoking programs called the "standard linkage convention." The convention is not enforced by the system; it's up to programmers to follow it.

IEFBR14 sets the condition code to zero and returns to the caller without performing any logic.

The sample program MYBR14 does the same thing. On entry, it saves the state of the general registers as of the point when the caller invoked it. Then it declares and initializes a base register.

After it finishes whatever work it does, the program restores the state of the registers, sets the condition code, and returns to the caller by branching on register 14, which by convention contains the return address (not enforced by the system). For a top-level program (named in an EXEC statement), control returns to JES.

So, this is a kind of push/pop sequence, but there's no explicit stack memory as such.

JES will act on any DD statements that are defined in the job step. The program itself never looks at the data sets. This has no direct analogue on Unix, Linux, or Windows.

WHAT YOU KNOW AND WHAT YOU CAN DO

- You can write JCL to execute the "do-nothing" utility IEFBR14 and DD statement parameters to allocate, catalog, uncatalog, and unallocate sequential data sets.

LABS 15-20: USE JCL AND THEN ANSIBLE TO CREATE DATA SETS

Follow the instructions for Labs 15-20 in labs/labs.md in the course repo. These labs provide practice in

- Looking up pertinent details in the documentation
- Writing JCL, running jobs, and reviewing job output
- Writing Ansible playbooks, running them, and reviewing returned results
- Building familiarity with different kinds of z/OS data sets

Optionally, they can also provide practice in

Developing context information and prompts for LLMs to assist with creating JCL and Ansible playbooks

Collaborating with team mates on tasks that may expose weaknesses in your own knowledge or skills

WHAT'S NEXT

- Working with Generation Data Groups